

Хук `useImperativeHandle`

■ Статус	Готово
----------	--------

Хук `useImperativeHandle`

У цьому розділі розглянемо рідко використовувані або нішеві React-хуки. Вони не є частиною обов'язкової бази, а скоріше додатковим матеріалом для розширення кругозору. Достатньо знати про їхнє існування та базове призначення. Глибоко занурюватися не обов'язково — повернетеся до них за потреби.

У React дані зазвичай передаються зверху вниз через пропси, а зворотний зв'язок — колбеками. Але іноді **батьківському компоненту потрібно напяму викликати методи дочірнього**. Наприклад:

- встановити фокус на інпут;
- скинути значення;
- запустити анімацію.

У таких випадках допомагає `useImperativeHandle`. Він дозволяє **визначити публічний API** компонента, доступний через `ref`. Це дає змогу

- приховати внутрішню реалізацію;
- дати батьківському компоненту лише необхідні методи;
- зробити дочірній компонент більш контрольованим.

Синтаксис

```
useImperativeHandle(ref, () => value, [dependencies]);
```

- `ref` — об'єкт посилання, який передається у компонент
- функція повертає публічний об'єкт
- залежності — при їх зміні об'єкт оновлюється

Приклад

Уявімо, що ми хочемо, щоб батьківський компонент міг і фокусувати, і очищати кастомний інпут.

```
import { useImperativeHandle, useRef } from 'react';

export type InputHandle = {
  focus: () => void;
  reset: () => void;
};

interface CustomInputProps {
  ref?: React.Ref<InputHandle | null>;
}

export default function CustomInput({ ref }: CustomInputProps) {
  const inputRef = useRef<HTMLInputElement | null>(null);

  useImperativeHandle(ref, () => ({
    focus() {
      inputRef.current?.focus();
    },
    reset() {
      if (inputRef.current) {
        inputRef.current.value = '';
      }
    },
  }));
}
```

```
    ));  
  
    return <input ref={inputRef} />;  
  }  
}
```

Батьківський компонент:

```
import { useRef } from 'react';  
import CustomInput, { type InputHandle } from './CustomInput';  
  
export default function App() {  
  const inputRef = useRef<InputHandle | null>(null);  
  
  return (  
    <>  
      <CustomInput ref={inputRef} />  
      <button onClick={() => inputRef.current?.focus()}>Focus input</button>  
      <button onClick={() => inputRef.current?.reset()}>Reset input</button>  
    </>  
  );  
}
```

Таким чином батьківський компонент отримує можливість **прямо керувати** дочірнім елементом.

Ось повний код прикладу в редакторі:

<https://stackblitz.com/edit/vitejs-vite-a51alycb?ctl=1&embed=1&file=src%2FApp.tsx&hideNavigation=1>

Для чого це потрібно?

- **Інкапсуляція** — зовні доступний тільки потрібний API

- **Гнучкість** — можна змінювати внутрішню реалізацію без зміни зовнішнього контракту
- **Контроль** — чітке визначення публічного інтерфейсу

Важливі моменти

- використовуйте `useImperativeHandle` лише тоді, коли справді потрібен прямий контроль
- якщо достатньо просто зчитати DOM-елемент, підійде звичайний `ref`
- усе, що ви віддаєте як публічний API, має бути ретельно продумане

Підсумок

`useImperativeHandle` дозволяє створити власний публічний інтерфейс дочірнього компонента через `ref`, роблячи його більш передбачуваним і контрольованим. Це корисно там, де потрібна точкова взаємодія з внутрішніми методами, але застосовуйте його лише за реальної потреби.